

מצביעים חכמים - Smart pointers

הכוונה ביצירת מצביעים חכמים, היא ליצור אובייקטים שהשימוש בהם יהיה דומה לשימוש במצביעים רגילים (טיפשים), הקיימים בשפה. היתרון בשימוש בהם יהיה במאפיינים הבאים של התנהגות:

- 🚩 **בנייה והריסה:** אנו יכולים לקבוע מה קורה כשהמצביע נוצר / נהרס. למשל – בבנייה ינתן לו ערך ברירת מחדל של 0, כדי למנוע הצבעה לא מבוקרת. הריסה – להעביר את האחריות למחיקת השטח המוצבע למצביע עצמו, עזר במניעה של דליפות זיכרון.
- 🚩 **העתקות בין מצביעים:** שליטה על מה שיתבצע. למשל – העתקה תגרור העתקה של השטח המוצבע, או אולי של המצביע עצמו? אולי נרצה למנוע העתקות בכלל?
- 🚩 **התייחסות:** מה קורה כשמתייחסים לאובייקט, דרך מצביע חכם? אנו נקבע זאת.

כדי להכריח את המשתמש להגדיר את הטיפוס המוצבע, יש ליצור את מחלקת מצביע חכם ע"י תבנית:

```
template<class T>
class SmartPtr
{
public:
    SmartPtr(T* realPtr = 0);
    ~SmartPtr();

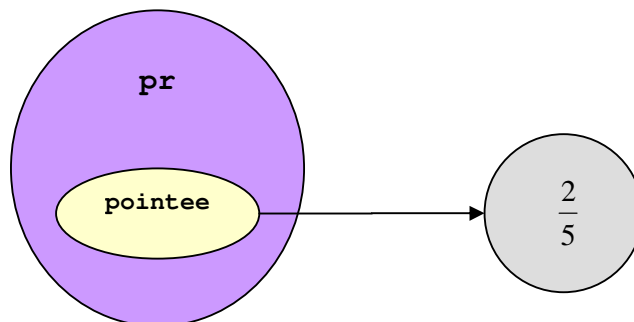
    SmartPtr(const SmartPtr& rhs);
    SmartPtr& operator=(const SmartPtr& rhs);

    T* operator->() const;
    T& operator*() const;

private:
    T *pointee;
};

SmartPointer<Rational> pr = new Rational(2,5);
cout<<pr->getNumerator()<<endl;    // usage of
                                     // SmartPointer::operator->

SmartPointer<int> pn = new int;
*pn = 5;                            // usage of
cout<<*pn<<endl;                    // SmartPointer::operator*
```



בנייה, השמה והריסה של מצביעים חכמים

בנייה אינה מסובכת. יש לכוון את המצביע הפנימי כך שיצביע על האובייקט, שניתן כפרמטר. אם לא ניתן אובייקט ממש אפשר לתת למצביע ערך 0 או להודיע על שגיאה.

העתקה, השמה והריסה הן יותר מסובכות. הנושא המשפיע הוא הבעלות. אם (ורק אם) מצביע חכם הוא כרגע הבעלים של האובייקט עליו הוא מצביע, הוא אחראי למחיקתו כשהורסים את המצביע החכם. זאת כמובן בהנחה שהאובייקט עליו מצביעים הוקצה דינאמית.

דוגמה למחלקה של מצביע חכם: בזמן השחרור יש למחוק את ההקצאה של האובייקט עליו מצביעים.

```
template<class T>
class auto_ptr
{
public:
    auto_ptr(T *ptr = 0): pointee(ptr) {}
    ~auto_ptr() { delete pointee; }
    ...

private:
    T *pointee;
};
```

זה תמיד יהיה נכון כשרק למצביע חכם אחד יש בעלות על האובייקט המוצבע.
מה יקרה בעת השמה / העתקה?

```
auto_ptr<Rational> ptr1(new Rational);

auto_ptr<Rational> ptr2 = ptr1;           // call to copy ctor;
                                           // what should happen?

auto_ptr<Rational> ptr3;

ptr3 = ptr2;                             // call to operator=;
                                           // what should happen?
```

אם העתקנו רק את המצביע, התוצאה תהיה שני מצביעים חכמים, שמצביעים לאותו אובייקט. כיוון שכל אחד מהם ימחק את האובייקט עליו הוא מצביע בפונקציה ההורסת שלו, התוצאה תהיה מצערת (נסיון למחוק אובייקט יותר מפעם אחת).

אפשרות אחרת היא להקצות אובייקט חדש, הוזה בערכו למקור. זה ימנע מצב בו מספר מצביעים חכמים מכילים מצביע המכוון לאותו אובייקט, כי לכל אחד יש אובייקט משלו.
פתרון זה יקר – הוא גורר יצירות והריסות רבות של אובייקטים. בנוסף – לא תמיד נדע איזה סוג אובייקט יש ליצור: אולי זהו טיפוס היורש מ-T?

ניתן לפתור את הבעייה אם נמנע אפשרות להעתקה ולהשמה.
זה פתרון די מגביל.

נראה את הפתרון הגמיש, שמיושם עבור המחלקה הסטנדרטית: בעת העתקה או השמה, יש **העברת בעלות** על האובייקט המוצבע.

כל מצביע חכם יכול להיות הבעלים של אובייקט אחד או של אף אובייקט.

לכל אובייקט יש מקסימום בעלים אחד.

```
template<class T>
class auto_ptr
{
public:
    ...

    auto_ptr(auto_ptr<T>& rhs);

    auto_ptr<T>& operator=(auto_ptr<T>& rhs);

    ...
};

template<class T>
auto_ptr<T>::auto_ptr(auto_ptr<T>& rhs)
{
    pointee = rhs.pointee;           // transfer ownership of
                                     // *pointee to *this

    rhs.pointee = 0;                 // rhs no longer owns
                                     // anything
}

template<class T>
auto_ptr<T>& auto_ptr<T>::operator=(auto_ptr<T>& rhs)
{
    if (this == &rhs)                // do nothing if this
        return *this;                // object is being assigned
                                     // to itself

    delete pointee;                  // delete currently owned
                                     // object; what will happen
                                     // if we fail to do so?

    pointee = rhs.pointee;           // transfer ownership of
    rhs.pointee = 0;                  // *pointee from rhs to *this

    return *this;
}
```

בשל העובדה כי העתקה גוררת העברת בעלות, העברת אובייקט מסוג מצביע חכם ע"י ערך היא מעשה שבדרך כלל מצטערים עליו:

```
// this function will often lead to disaster
void printListNode(ostream& s, auto_ptr<ListNode> p)
{
    s << *p;
}

int main()
{
    auto_ptr<ListNode> ptn(new ListNode);

    ...

    printListNode(cout, ptn);        // pass auto_ptr by value

    ...

}
```

כשהפרמטר של הפונקציה מאתחל, ע"י קריאה לפונקציית העתקה, הבעלות על האובייקט שהוצבע ע"י ptn עוברת ל-p. עם סיום הפונקציה p נהרס, והורס עמו (בפונקציה ההורסת של המצביע החכם) את האובייקט שהיה מוצבע ע"י ptn. כרגע ptn מצביע ל-NULL, וכל נסיון להתייחס למה ש(היה) מוצבע על ידו יגרור הפתעות. אם כן, העברת מצביע חכם כערך תעשה אם ורק אם ברצונך להעביר את הבעלות על האובייקט המוצבע לפרמטר של הפונקציה.

בדרך כלל נעביר מצביע חכם ע"י ייחוס:

```
// this function behaves much more intuitively
void printListNode(ostream& s, const auto_ptr<ListNode>& p)
{
    s << *p;
}
```

בפונקציה זו p אינו אובייקט, לכן אין אתחול שלו ע"י פונקציה. כשנמסור הפעם את ptn לפונקציה תשמר הבעלות שלו על האובייקט. ניתן יהיה אחר הקריאה להתייחס לאובייקט המוצבע ע"י ptn ללא חשש.

כל זאת ועוד: ישנה הסיבה הרגילה להעביר פרמטר ע"י ייחוס ולא כערך: יעילות.

שימו לב לצורה בה כתובים אופרטור ההשמה ופונקציית ההעתקה: אנו רגילים לקבל כפרמטר ייחוס קבוע. פה אין זה כך. זה בגלל שפה משנים את המקור ממנו מעתיקים, לצורך ההעתקה. לכן אין אפשרות שהמקור יהיה קבוע. אנו גוזלים ממנו את הבעלות על האובייקט שמוצבע על ידו. זה דבר לא רגיל: אובייקט שמשנתה כאשר הוא משמש כמקור של השמה / העתקה.

הריסה של מצביע חכם:

```
template<class T>
SmartPtr<T>::~~SmartPtr()
{
    if (*this owns *pointee)
    {
        delete pointee;
    }
}
```

לא תמיד יש צורך לבדוק לפני המחיקה: יכול להיות מימוש בו כל מצביע חכם מצביע על אובייקט משלו ואז חייבים למחוק. או מימוש בו כל מצביע חכם מצביע על אובייקט חיצוני שתמיד אינו שלו ואז לעולם אין למחוק. תלוי במימוש. במימוש האחרון שראינו הבדיקה תהיה

```
if (pointee)
{
    delete pointee;
}
```

מימוש אופרטורים של התייחסות (Dereference)

נתייחס לפונקציות הבאות:

`operator*` 
`operator->` 

הראשונה אמורה להחזיר את האובייקט המוצבע:

```
template<class T>
T& SmartPtr<T>::operator*() const
{
    perform "smart pointer" processing;

    return *pointee;
}
```

ייתכן ויש ליצור אובייקט חוקי שהוא זה שיוחזר, כפעולה מקדימה. אין זה מובן מאליו כי ניתן להחזיר את ערכו של האובייקט, אם אין אנו מצביעים אל שום אובייקט...

שימו לב לכך שמפעולה זו מוחזר ייחוס. מה יקרה אם נחזיר ערך? קודם כל, כמובן שהקומפיילר ירשה זאת. אך אם באמת הצבענו על אובייקט מסוג היורש מ-`T`, ולא `T` ממש? אז טיפוס ההחזרה לא יהיה נכון:

☹. האובייקט ייחתך להיות מטיפוס האב.

☹. בהפעלת פונקציות וירטואליות – לא תופעל הפונקציה הנכונה לטיפוס המקורי.

כל זאת ועוד: ישנה הסיבה הרגילה להחזיר ייחוס ולא ערך: יעילות.

מה יקרה אם מישהו יחליט להפעיל את הפונקציה הזו עבור ערך `NULL`? כל מה שנרצה. ביכולתנו להחליט. למשל – לזרוק חריגה. העיקר הוא למנוע גישה לערך שאיננו, מה שכנראה ימוטט את התכנית.

הפונקציה `operator->`:

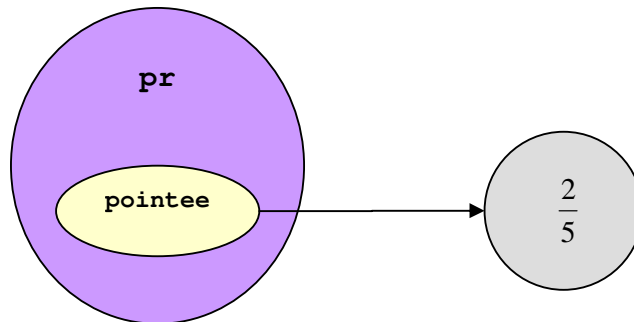
```
template<class T>
T* SmartPtr<T>::operator->() const
{
    perform "smart pointer" processing;

    return pointee;
}
```

כיוון שמוחזר מצביע, אין כל בעייה להפעיל פונקציות (כולל וירטואליות) בצורה הרצויה:

```
SmartPointer<Rational> pr = new Rational(2,5);  
cout<<pr->getNumerator()<<endl;
```

```
Person* ptrp;  
...  
SmartPointer<Person> pp = ptrp;  
cout<<pp->getAddress()->getCountry()<<endl;
```



בדיקה לריקנות (Nullness)

כיצד ניתן לבדוק, אם מצביע חכם מצביע כרגע ל- NULL? זכרו: אחת המטרות הייתה קלות בשימוש, כמו עם פויינטרים רגילים:

```
SmartPtr<ListNode> ptn;

...

if (ptn == 0) ...           // error!
if (ptn) ...                // error!
if (!ptn) ...               // error!
```

אכן מגבלה רצינית. יש שתי אפשרויות לפתרון:

🚧 להגדיר פונקציית מחלקה הנקראת `isNull`, המחזירה את תוצאת ההשוואה ל- `NULL`.
🚧 להגדיר אופקטור המרה (`implicit conversion operator`).

היות ומטרה עיקרית פה היא יכולת שימוש נוחה במצביע חכם, ננסה את אפשרות 2:

```
template<class T>
class SmartPtr
{
public:
    ...
    operator void*();           // returns 0 if the smart
    ...                         // ptr is null, nonzero
};                               // otherwise

SmartPtr<ListNode> ptn;

...

if (ptn == 0) ...              // now fine
if (ptn) ...                   // also fine
if (!ptn) ...                  // fine
```


לפונקציות המרה יש גם חסרונות.
חסרונות משמעותיים:

```
SmartPtr<Apple> pa;  
SmartPtr<Orange> po;  
  
...  
  
if (pa == po) ...           // this compiles!
```

אבל למה? הרי לא סיפקנו אופרטור השוואה (operator==).
אז מה. אופרטור כזה קיים כברירת מחדל עבור מצביעים רגילים. הקומפיילר רואה אפשרות לבצע את
הקריאה בהצלחה תוך המרת שני המצביעים החכמים למצביע כללי, ואז השוואתם. התוצאה, כמו פה,
יכולה להיות למשל השוואה בין טיפוסים שונים לחלוטין.
אז כמה חכם המצביע החכם שלנו עכשיו?

ניתן לספק פשרה באמצעות operator!. פונקציה זו, באופן לוגי, תחזיר true אם ורק אם המצביע
הוא NULL. זה יאפשר, לפחות באופן חלקי, שימוש נוח:

```
template<class T>  
class SmartPtr  
{  
public:  
    ...  
    bool operator!() const;           // returns true if and only  
    ...                               // if the smart ptr is null  
};  
  
SmartPtr<ListNode> ptn;  
...  
  
if (!ptn)                             // fine  
{  
    ...                               // ptn is null  
}  
else  
{  
    ...                               // ptn is not null  
}  
  
if (ptn == 0) ...                     // still an error  
  
if (ptn) ...                          // also an error
```

עדיין קיים הסיכון:

```
SmartPtr<Apple> pa;  
SmartPtr<Orange> po;  
  
...  
  
if (!pa == !po) ...           // this compiles
```

היות ולא רבים המתכנתים שיכתבו כך, ניתן להסתפק במה שיש: סיפקנו תמיכה סבירה עבור בדיקה לריקנות של המצביע החכם. הוא עדיין לא גאון הדור, אבל כבר לא אידיוט מושלם.

המרת מצביעים חכמים למצביעים רגילים (טיפשים)

לעתים יש צורך להפעיל פונקציה, המקבלת מצביע רגיל. זה יכול לקרות למשל בשימוש בפונקציה שיש לנו יכולת להשתמש בה, אך לא לשנות אותה. נניח כי בשאר המקרים אנחנו משתמשים במצביעים חכמים.

```
class Person { ... };
```

```
void update(Person* pp);
```

```
SmartPtr<Person> pp;
```

```
...
```

```
update(pp); // error!
```

הקריאה לא תצליח, כי אין כל אפשרות להמיר `SmartPtr<Person>` ל-`Person*`.
תוכל להצליח ע"י

```
update(&*pp); // gross, but legal
```

לא ממש יפה לדרוש מכל מי שכותב בתכנית הראשית לכתוב כך.

הצורה המסודרת תהיה להוסיף פונקציית המרה למצביע (רגיל) מטיפוס `T`:

```
template<class T> // as before
class SmartPtr
{
public:
    ...
    operator T*() { return pointee; }
    ...
};
```

```
SmartPtr<Person> pp;
```

```
...
```

```
update(pp); // this now works
```

מה עוד נוכל לרצות? הוספת פונקציה זו מחסלת גם את הבעייה שהייתה בבדיקה ל-NULL:

```
if (pp == 0) ... // fine, converts pp
                  // to a Person*

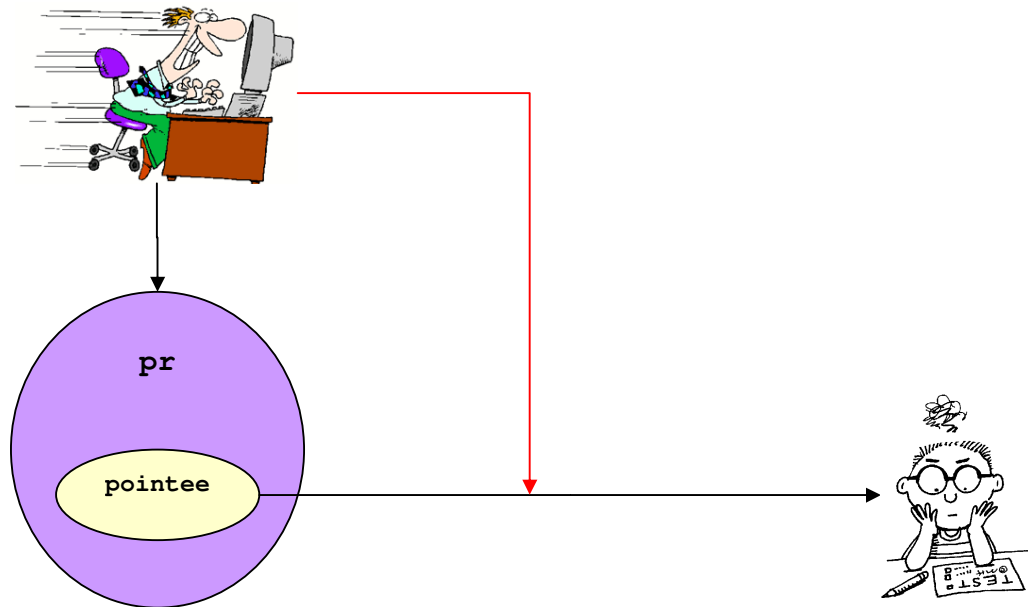
if (pp) ... // ditto

if (!pp) ... // ditto
```

הפונקציה החדשה יוצרת בעייה אחרת. בעייה עקרונית.
היא מאפשרת לעקוף את המצביע החכם ולבצע כל מה שרוצים דרך המצביע הרגיל:

```
void processPerson(SmartPtr<Person>& pp)
{
    Person *rawPersonPtr = pp; // converts SmartPtr<Person>
                                // to Person*

    use rawPersonPtr to modify the Person;
}
```



גם אם נספק אופרטור להמרה מפורשת, עדיין לא נוכל לעבוד עם מצביע חכם ממש כמו מצביע רגיל. זאת כיוון שפונקציית ההמרה הוגדרה על ידינו, והקומפיילר לא יאפשר יותר מהמרה אחת כדי להגיע לטיפוס הנכון:

```
class String
{
public:
    String(const char *pc);
    ...
};

int strcmp(const String& s1, const String& s2);

char *c1, *c2;

...

strcmp(c1, c2);           // fine, both pointers are converted
                           // to String objects

SmartPtr<char> pt1, pt2;

...

strcmp(pt1, pt2);         // error! No way to convert pt1 and
                           // pt2 to String objects
```

זו שגיאה, כיוון שכדי להמיר `SmartPtr<char>` ל-`String`, יש לבצע רצף של שתי המרות מוגדרות על ידינו:

1. `SmartPtr<char> ← char*` ע"י אופרטור המרה של מצביע חכם.
2. `String ← char*` ע"י פונקציה בונה, שפה משמשת כפונקציית המרה.

הקומפיילר לא מרשה יותר מהמרה אחת.
רק אחת ודי.

בתור בונוס על מה שלא קיבלנו, פונקציות המרה פותחות תיבת פנדורה לכוז כתיבה:

```
SmartPtr<Person> pt = new Person;

...

delete pt;           // Now this, of all things, compiles.
                     // Unfortunate, isn't it?
```

לא יכול להיות. pt הוא אובייקט. לא ניתן למחוק אובייקט.
מצביעים ניתן למחוק.
אובייקטים לא.

נכון. אבל הקומפיילר מחפש דרך בה הקריאה תצליח והוא מוצא את הפונקציה

```
SmartPtr::operator T*()
{
    return pointee;
}
```

שמחזירה מצביע מטיפוס T.

מה אמרנו?
אה, כן. מצביעים ניתן למחוק.
במקרה הזה מחיקת המצביע גם מוחקת כל סיכוי להתנהגות נורמלית של התכנית.

אם המצביע היה הבעלים של האובייקט המוצבע, אובייקט זה יימחק פעמיים:
1. בקריאה ל- delete.
2. בעת סיום הבלוק בו מוגדר המצביע, אז נקראת הפונקציה ההורסת שלו.

אם המצביע לא היה הבעלים, מישו אחר היה:
← אם הבעלים הינו אותו אחד שקרא ל- delete, ניצלנו.
← אם לא, הבעלים לא יודע על המחיקה, לכן בעתיד ימחקו בעצמו. שוב, זהו מקרה של נסיון למחוק את אותו אובייקט פעמיים. אנו יודעים שלא ניתן לדעת מה יקרה אז.

הבעייה היא בעייה עקרונית. רצינו מצביע חכם, שבקלות השימוש בו יהיה דומה ממש למצביע רגיל. ככל שהוא דומה, המשתמשים עלולים לשכוח את ההבדל, וללכצע טעויות כמו מחיקה, מחשש לגרום לדליפת זיכרון. ובכלל, זה לא כל כך מוזר שהם עלולים לקרוא ל- delete אחרי שהם קראו ל- new.

כלל המפתח הוא: לא לספק אופרטורים להמרה בלתי מפורשת (implicit conversion operators) ממצביע חכם למצביע רגיל, כל עוד אין סיבה טובה לזה.
סיבה ממש טובה.

מצביעים חכמים והמרה מבוססת ירושה

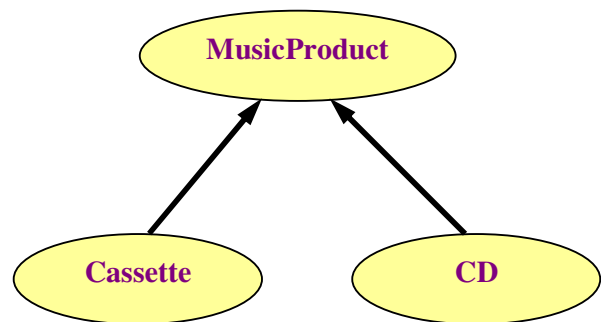
נראה את ההיררכיה הבאה:

```
class MusicProduct
{
public:
    MusicProduct(const string& title);
    virtual void play() const = 0;
    virtual void displayTitle() const = 0;
    ...
};
```

```
class Cassette: public MusicProduct
{
public:
    Cassette(const string& title);
    virtual void play() const;
    virtual void displayTitle() const;
    ...
};
```

```
class CD: public MusicProduct
{
public:
    CD(const string& title);
    virtual void play() const;
    virtual void displayTitle() const;
    ...
};
```

```
void displayAndPlay(const MusicProduct* pmp, int numTimes)
{
    for (int i = 1; i <= numTimes; ++i)
    {
        pmp->displayTitle();
        pmp->play();
    }
}
```



```

Cassette *funMusic = new Cassette("Radio Ga Ga");
CD *nightmareMusic = new CD("Radio Gu Gu");

displayAndPlay(funMusic, 10);
displayAndPlay(nightmareMusic, 0);

```

עד כאן אין הפתעות.
ננסה להחליף את המצביעים הרגילים בכאלו חכמים:

```

void displayAndPlay(const SmartPtr<MusicProduct>& pmp, int numTimes);

SmartPtr<Cassette> funMusic(new Cassette("Radio Ga Ga"));
SmartPtr<CD> nightmareMusic(new CD("Radio Gu Gu"));

displayAndPlay(funMusic, 10);           // error!
displayAndPlay(nightmareMusic, 0);      // error!

```

מה עכשיו לא טוב?

הבעיה היא שניסינו להתייחס לקשר של ירושה שאינו קיים.
ניתן לבצע את ההמרות:

```

MusicProduct* ← Cassette* 🚫
MusicProduct* ← CD* 🚫

```

זאת מכיוון שניתן להמיר מצביע מטיפוס יורש למצביע ממחלקת הבסיס.
אבל אנחנו ניסינו את ההמרות:

```

SmartPtr<MusicProduct> ← SmartPtr<Cassette> 🚫
SmartPtr<MusicProduct> ← SmartPtr<CD> 🚫

```

אלו טיפוסים שונים לחלוטין.

אין כל קשר ביניהם.

נכון, הם נוצרו מאותה תבנית (template). אז מה. בזמן קומפילציה נוצר קוד נפרד עבור כל אחד מטיפוסים אלו, בדיוק כמו שנוצר עבור טיפוסים אחרים שאין קשר ביניהם. לרבות ירושה.

אבל יש פתרון, בצורה של אופרטור להמרה.
יש לספק כזה עבור המחלקות `SmartPtr<Cassette>`, `SmartPtr<CD>`. זאת במטרה
לאפשר המרה לטיפוס `SmartPtr<MusicProduct>`.

```
class SmartPtr<Cassette>
{
public:
    operator SmartPtr<MusicProduct>()
    { return SmartPtr<MusicProduct>(pointee); }

    ...

private:
    Cassette *pointee;
};

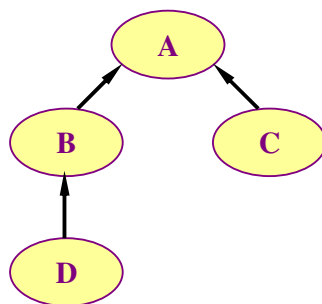
class SmartPtr<CD>
{
public:
    operator SmartPtr<MusicProduct>()
    { return SmartPtr<MusicProduct>(pointee); }

    ...

private:
    CD *pointee;
};
```

זה עובד, אם כי צורם:

יש להגדיר באופן ספציפי את האופרטורים להמרה, עבור מקרים מיוחדים של מצביעים חכמים
שם הם נחוצים. אם יש להגדיר באופן ספציפי, אנו סותרים את כל הרעיון של יצירה ע"י תבנית
(template).
בכל מקרה בו נרצה המרה כזו, בדומה להמרה המתבססת על ירושה, נצטרך, בכל מצביע חכם
מטיפוס כלשהו, לכתוב אופרטורים להמרה לא רק למחלקה ממנה ירש ישירות. יש לכתוב
אופרטורים עבור כל ההיררכייה שמעליו:



במחלקת `SmartPtr<D>` יש להגדיר המרות עבור `SmartPtr<B>` ועבור
`SmartPtr<A>`.

זכרו: הקומפילר לא מוכן לבצע דרך של שתי המרות מוגדרות על ידינו כדי שהקריאה תצליל.
לכן לא ניתן להמיר: `SmartPtr<D> ← SmartPtr<B> ← SmartPtr<A>`.

רצוי היה שהקומפיילר יכתוב עבורנו את כל הפונקציות הנ"ל. לשם כך יש לאפשר יצירה שלהן ע"י תבנית: אלו הן פונקציות מחלקה מבוססות תבנית, member templates או member function templates.

```
template<class T>                // template class for smart
class SmartPtr                  // pointers-to-T objects
{
public:
    SmartPtr(T* realPtr = 0);

    T* operator->() const;
    T& operator*() const;

    template<class newType>      // template function for
    operator SmartPtr<newType>() // implicit conversion ops.
    {
        return SmartPtr<newType>(pointee);
    }

    ...
};

void displayAndPlay(const SmartPtr<MusicProduct>& pmp, int howMany);

SmartPtr<Cassette> funMusic(new Cassette("Radio Ga Ga"));
SmartPtr<CD> nightmareMusic(new CD("Radio Gu Gu"));

displayAndPlay(funMusic, 10);        // used to be an error
displayAndPlay(nightmareMusic, 0);   // used to be an error
```

כעת, בקריאה כזו

```
displayAndPlay(funMusic, 10);
```

הקומפיילר מחפש דרך לבצע את ההמרה של `funMusic` מטיפוס `SmartPtr<Cassette>` ← `SmartPtr<MusicProduct>` (הראשון הוא הטיפוס עמו קראנו לפונקציה, השני הוא הפרמטר בה). הוא מחפש:

1. פונקציה בונה בעלת ארגומנט בודד במחלקת `SmartPtr<MusicProduct>`, כזו שמקבלת טיפוס `SmartPtr<Cassette>`. אין כזו פונקציה.
2. אופרטור המרה במחלקת `SmartPtr<Cassette>`, כזה שיניב אובייקט מטיפוס `SmartPtr<MusicProduct>`. גם החיפוש הזה נכשל.
3. פונקציית מחלקה מתבנית, כזו שיכולה להניב אחת מהאפשרויות 1 או 2. פונקציה כזו נמצאת מבחינתו במחלקת `SmartPtr<Cassette>`. הוא מחולל את הפונקציה הבאה:

```
SmartPtr<Cassette>::operator SmartPtr<MusicProduct>()  
{  
    return SmartPtr<MusicProduct>(pointee);  
}
```

האם זה יתקמפל?

בעצם אנו קוראים לפונקציה הבונה של `SmartPtr<MusicProduct>` עם `pointee` כפרמטר. השאלה היא, איפה, האם ניתן לבנות אובייקט `SmartPtr<MusicProduct>` ע"י מצביע מסוג `Cassette`. הרי הפונקציה הבונה מצפה לקבל מצביע מטיפוס `MusicProduct`.

זוהי כבר המרה קיימת בשפה: המרה בין סוגי מצביעים. כיוון שניתן להמיר מצביע כלפי מעלה בהיררכיית הירושה, זה יעבוד.

בעצם, זה יעבוד עבור כל המרה של מצביעים שיש בשפה. ההמרה צפויה להצליח, אם ורק אם המצביעים שבתוך המצביעים החכמים שמעורבים בהמרה יכולים להמרה הטבועה בשפה.

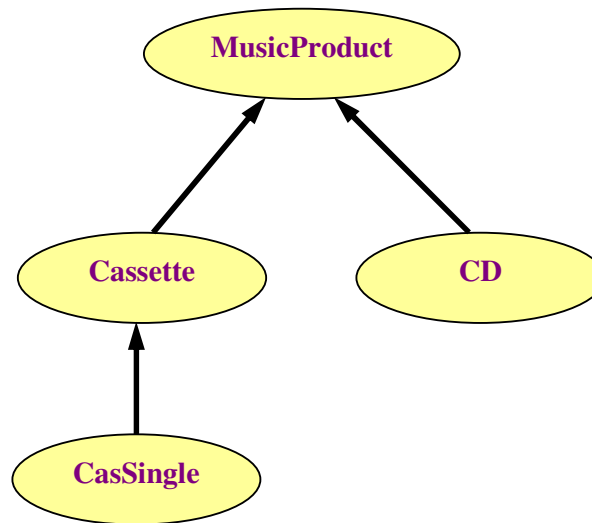
😊 . הרווחנו את היכולת להמיר מתי שרצינו.

😊 . שמרנו על ההגבלות הנכונות למצביעים רגילים, גם עבור מצביעים חכמים.

כל זה נותן לנו את ההתנהגות שרצינו.

כמעט.

נניח שנרצה להוסיף מחלקה חדשה להיררכייה:



```
template<class T>          // as above, including member tem-
class SmartPtr { ... };   // plate for conversion operators

void displayAndPlay(const SmartPtr<MusicProduct>& pmp, int howMany);

void displayAndPlay(const SmartPtr<Cassette>& pc, int howMany);

SmartPtr<CasSingle> dumbMusic (new CasSingle("Innuendo"));

displayAndPlay(dumbMusic, 1);          // error!
```

פה חפפנו את הפונקציה החיצונית, המציגה ומנגנת מוצר מוסיקלי. כתבנו פונקציה המקבלת `SmartPtr<Cassette>`, בנוסף לזו שמקבלת `SmartPtr<MusicProduct>`. כשאנו קוראים לפונקציה עם `SmartPtr<CasSingle>`, אנו מצפים שזו שתיבחר היא `SmartPtr<Cassette>`, כי `CasSingle` יורשת ישירות מ-`Cassette`, ורק בעקיפין מ-`MusicProduct`. כך זה עובד עבור מצביעים רגילים.

זה לא המצב עם מצביעים חכמים. ההמרה שלנו מתבצעת הודות לפונקציית המרה. מבחינת הקומפיילר, כל פונקציות ההמרה שיכולות לבצע אותה טובות באותה מידה. לכן הקריאה היא דו-משמעית. ההמרה מטיפוס `SmartPtr<CasSingle>` לטיפוס `SmartPtr<Cassette>`, אינה טובה יותר מההמרה ל-`SmartPtr<MusicProduct>`.

עוד שני חסרונות לפונקציות המרה של מצביעים חכמים:
תמיכה בנושא של פונקציות מחלקה מבוססות תבנית עדיין נדירה. 
השיטה לא מספקת פתרון פשוט, קל להבנה ולתחזוקה. 

אבל מה שבאמת מעניין הוא איך לגרום למצביעים חכמים להתנהג כמו מצביעים רגילים בנושא של המרות מבוססות ירושה בין טיפוסים.

התשובה היא שלא ניתן.
אם יש צורך, אפשר לספק פונקצית המרה מתבנית ובמקומות של דו-משמעות לבצע casting.

מצביעים חכמים קבועים (const)

כזכור, עבור מצביעים רגילים: יש ארבע אפשרויות של const:

```
CD goodCD("The figure one cuts");

const CD *p;                // p is a non-const pointer
                             // to a const CD object

CD * const p = &goodCD;     // p is a const pointer to
                             // a non-const CD object;
                             // because p is const, it
                             // must be initialized

const CD * const p = &goodCD; // p is a const pointer to
                             // a const CD object
```

היינו רוצים את אותה גמישות עם מצביעים חכמים.

הבעיה היא שמצביע חכם הוא לא מצביע.

הוא אובייקט.

הוא אובייקט, שמכיל מצביע לאובייקט. לכן יש רק מקום אחד בו ניתן לשים לו const. במקום זה, זה אומר שהמצביע הוא קבוע, האובייקט המוצבע לא:

```
const SmartPtr<CD> p = &goodCD;    // p is a const smart ptr
                                    // to a non-const CD object
```

אולי ניתן לקבע כך את האובייקט?

```
SmartPtr<const CD> p = &goodCD;    // p is a non-const smart ptr
                                    // to a const CD object
```

הנה הם באים: ארבעת המינים של מצביעים חכמים קבועים:

```
SmartPtr<CD> p;                  // non-const object,
                                // non-const pointer

SmartPtr<const CD> p;            // const object,
                                // non-const pointer

const SmartPtr<CD> p = &goodCD;  // non-const object,
                                // const pointer

const SmartPtr<const CD> p = &goodCD; // const object,
                                // const pointer
```

פעם אמרנו, שהיה רצוי כי מצביעים חכמים יתנהגו דומה לרגילים.
תכונה הגיונית שקיימת בשפה אומרת שניתן לעלות בדרגת הקביעות ללא צורך בהמרה כלשהי. זאת אומרת, שניתן להשים ממצביע לא-קבוע למצביע קבוע:

```
CD *pCD = new CD("Balls To Picasso");  
  
const CD * pConstCD = pCD; // fine
```

```
SmartPtr<CD> pCD = new CD("One By One");  
  
SmartPtr<const CD> pConstCD = pCD; // fine?
```

לקומפיילר אין רחמים. יכולת ניתוח כן, רחמים לא.
מבחינתו, SmartPtr<CD> ו-SmartPtr<const CD> הם שני טיפוסים שונים. כיוון שכן, אין כל סיבה לאפשר לנו לבצע ביניהם השמות / העתקות.

כדי להוסיף את האפשרות הנחשקת, יש לספק פונקציה, שיכולה להמיר SmartPtr<CD> ← SmartPtr<const CD>. למי שיש קומפיילר התומך בפונקציות מחלקה מתבנית (member templates), ניתן להשתמש בטכניקה שהוצגה פה קודם, כדי ליצור באופן אוטומטי את טיפוס ההמרה הדרוש, ברגע מסויים.

פונקציות כאלה, כמו שכבר נאמר, יתמכו בכל המרה המוכרת עבור מצביעים רגילים. גם עבור המרה של מצביע לא-קבוע ← מצביע קבוע.

יש כמה חוקים מוכרים להמרות כאלה:
✚ ניתן להמיר מצביע עם const (לאובייקט או למצביע) למצביע ללא const.
✚ כל מה שניתן לבצע עם מצביע const, ניתן לביצוע עם מצביע non-const. אך לא להיפך.
עבור מצביע לא-קבוע ניתן לבצע עוד פעולות – למשל השמה אליו.

החוקים נשמעים מוכרים?
אם כן, זה לא דו"ה-וו. זה בגלל ירושה ציבורית.
✚ ניתן להמיר אובייקט מהמחלקה הנגזרת לאובייקט ממחלקת הבסיס.
✚ כל שניתן לבצע עם אובייקט ממחלקת הבסיס, ניתן גם עם אובייקט מהמחלקה הנגזרת. אך לא להיפך.

את הדימיון הזה ניתן לנצל במימוש מצביעים חכמים.

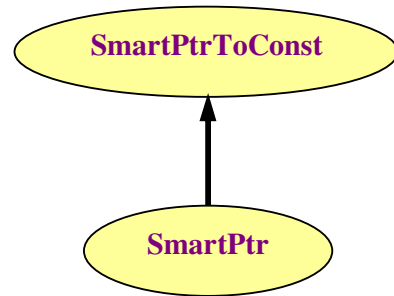
```
template<class T>
class SmartPtrToConst
{
    ...

```

```
protected:
```

```
    union
    {
        const T* constPointee;    // for SmartPtrToConst access
        T* pointee;               // for SmartPtr access
    };
};
```

```
template<class T>
class SmartPtr : public SmartPtrToConst<T>
{
    ...                          // no data members
};
```



אובייקט של מצביע חכם קבוע ישתמש במצביע הקבוע.
אובייקט של מצביע חכם רגיל ישתמש במצביע הרגיל.

בכל מקרה, עבור כל אחד משני הסוגים של מצביעים חכמים שכאן, אובייקט מכיל רק מצביע אחד מבחינת גודל: זה היתרון ב-union. המקום שאתה תופס הוא לפי גודל המשתנה אליו התייחסת. יש גם סיכון: צריך לממש בזהירות את פונקציות המחלקה, כך שיגשו עבור כל מחלקה למצביע הנכון.

```
SmartPtr<CD> pCD = new CD("War of the Worlds");
```

```
SmartPtrToConst<CD> pConstCD = pCD;    // fine
```


מילה לסיום

האם היתרונות בשימוש במצביעים חכמים שווים את הטרחה, שהם גוררים אחריהם?

עוד נראה מקרים מיוחדים בהם הם שימושיים במיוחד. לעומת זאת, הם קשים יותר לשימוש והבנה. יש בעיות לתמוך בהמרות למצביעים רגילים, בדיקות לריקנות וכו'. בנוסף, לא ניתן לממש מצביע חכם שיכול לשמש לכל דבר באופן הכי כללי, כמו מצביע רגיל.